

Informe

Práctica 3: Calidad de la visualización. Checks.

Visualización

Pablo de la Fuente Rodríguez
alu0101336152@ull.edu.es

San Cristóbal de La Laguna, 10 de marzo de 2026

Índice

Introducción.....	3
Análisis del script de ejemplo y funcionamiento de los Asset Checks.....	3
¿Qué hace el asset en el script?.....	3
¿Qué verificación se ha hecho?.....	3
Estructura del fichero definitions.py.....	3
¿Qué assets están en el manifiesto?.....	4
¿Qué checks están en el manifiesto?.....	4
Ejecución del proyecto en Dagster y validación del check.....	4
Forzado del fallo del check y análisis del resultado.....	5
Implementación de checks en el proyecto de rentas.....	7
Verificación del formato de los códigos territoriales.....	8
Verificación de ausencia de valores nulos en datos clave.....	9
Verificación de continuidad temporal.....	10
Control del número de categorías representadas.....	11
Verificación de coherencia del eje vertical.....	12
Forzado de fallo de un check.....	13

Introducción

En esta práctica se continúa trabajando con el proyecto de visualización sobre la distribución de rentas en Canarias, incorporando controles de calidad mediante el uso de *Asset Checks* en Dagster. El objetivo es asegurar que los datos, las transformaciones realizadas y las visualizaciones generadas cumplen una serie de requisitos básicos de coherencia, integridad y diseño.

Para ello, se añaden comprobaciones automáticas que permiten detectar errores en los datos o en las visualizaciones, como nombres mal escritos, valores incorrectos o configuraciones que puedan distorsionar la gráfica. Con esto se consigue que el pipeline no solo genere resultados, sino que también verifique que todo esté correcto antes de darlo por válido, mejorando así la calidad y la coherencia del proyecto.

Adjunto al final de este informe el enlace al repositorio de GitHub de la asignatura, donde se puede acceder a la carpeta correspondiente a esta práctica. En ella se encuentran todos los ficheros en Python utilizados, los Asset Checks implementados, las imágenes generadas durante la ejecución del proyecto y este mismo informe → [Enlace repositorio de GitHub](#).

Análisis del script de ejemplo y funcionamiento de los Asset Checks

En este apartado se analiza el script de ejemplo `test_checks.py`, proporcionado por la profesora, cuyo objetivo es mostrar el funcionamiento básico de los Asset Checks en Dagster.

¿Qué hace el asset en el script?

En primer lugar, se define el asset `islas_raw`, que carga un archivo CSV con datos y devuelve el DataFrame sin aplicar transformaciones adicionales. Este asset representa los datos en bruto dentro del pipeline y constituye el punto de partida sobre el que se aplican las verificaciones de calidad (checks).

¿Qué verificación se ha hecho?

Sobre este asset se define el check `check_estandarizacion_islas`, cuya finalidad es comprobar que los nombres de las islas estén escritos siempre de la misma forma. Para ello, se compara el número de categorías únicas originales con el número de categorías después de aplicar una normalización sencilla usando `capitalize()`, para poder realizar la comprobación.

Si el número coincide, significa que no hay diferencias en mayúsculas o minúsculas y el check se da por válido. Sin embargo, si al normalizar cambian las categorías, el check falla, lo que indica que existen nombres escritos de manera inconsistente. Este tipo de error puede generar problemas en la visualización, como categorías duplicadas o una leyenda incorrecta.

Estructura del fichero `definitions.py`

En este apartado se analiza la función del fichero `definitions.py`, que actúa como punto central de configuración del proyecto en Dagster. Su objetivo es registrar los elementos que forman parte del pipeline, de manera que la herramienta pueda reconocerlos y mostrarlos en su interfaz.

En este caso, en el fichero `definitions.py` se carga el archivo `test_checks`, donde están definidos el asset y el check de dicho fichero. De esta forma, Dagster puede reconocerlos y ejecutarlos dentro del proyecto.

¿Qué assets están en el manifiesto?

En el manifiesto del proyecto se encuentra registrado el asset `islas_raw`, ya que es el asset definido en el archivo `test_checks.py` y cargado mediante `load_assets_from_modules()`.

¿Qué checks están en el manifiesto?

En el manifiesto del proyecto se encuentra registrado el check `check_estandarizacion_islas`, ya que es el check definido en `test_checks.py` y cargado mediante `load_asset_checks_from_modules()`.

En resumen, el fichero `definitions.py` permite que Dagster reconozca el asset `islas_raw` y el check `check_estandarizacion_islas`, haciendo posible su materialización y validación desde la interfaz web de Dagster.

Ejecución del proyecto en Dagster y validación del check

Una vez configurado correctamente el fichero `definitions.py`, adaptándolo a la estructura de directorios de mi proyecto, se procede a ejecutar el proyecto utilizando el comando:

```
None  
> dagster dev -f src/definitions.py
```

Una vez ejecutamos el comando, ya tenemos la interfaz de Dagster cargada, y desde el apartado de lineage podemos observar el grafo de assets y sus checks asociados, en este caso solamente tenemos el asset `islas_raw` y su respectivo check, tal y como comentamos anteriormente.

A continuación, lo materializamos. Al hacerlo, Dagster ejecuta automáticamente tanto el asset como el check vinculado a él.

Tras la ejecución, el check `check_estandarizacion_islas` aparece validado correctamente en la interfaz, mostrando el estado “1/1 Passed”. Esto indica que los datos cumplen la condición definida, es decir, que los nombres de las islas están escritos de forma consistente y no generan categorías duplicadas.

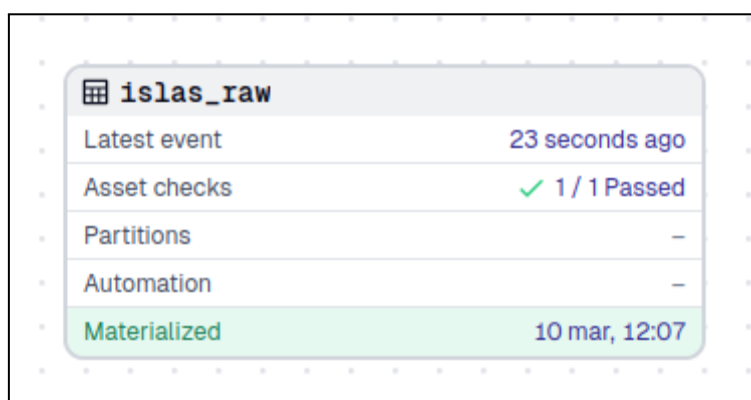
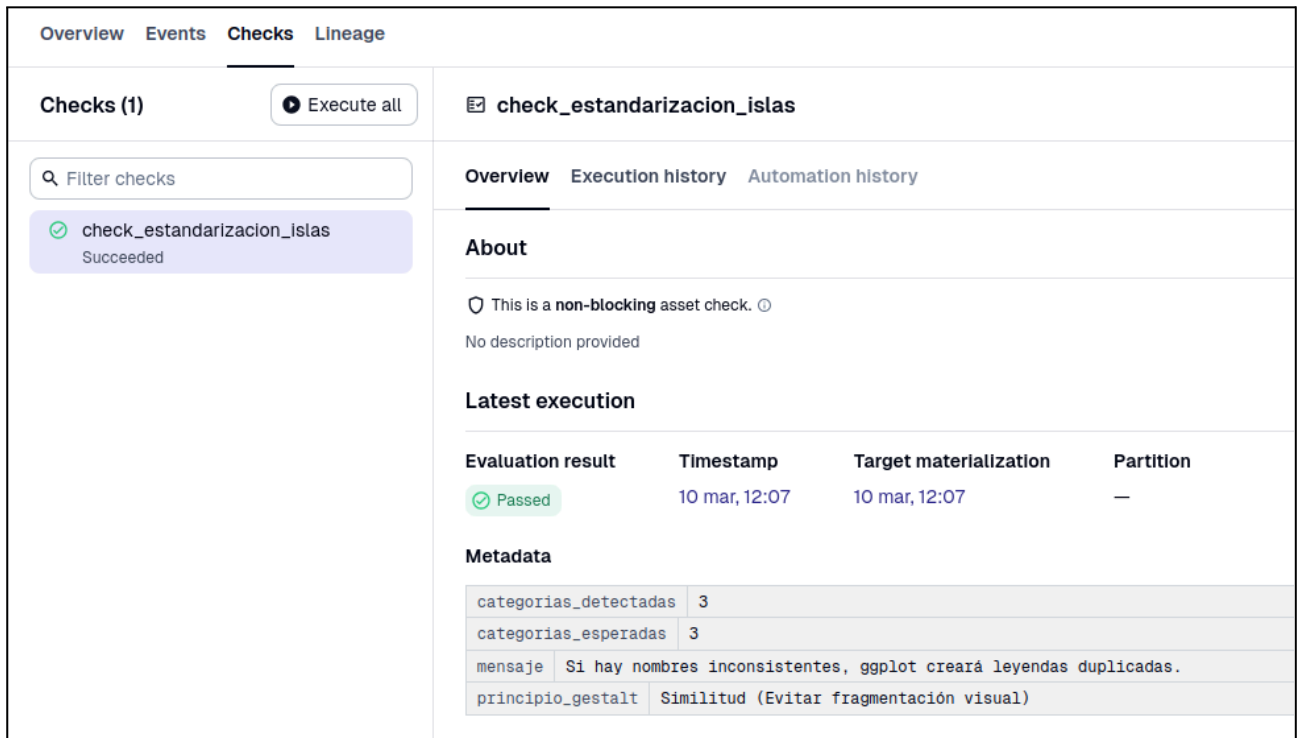


Figura 1. Resultado de la materialización del asset `islas_raw` con el check validado correctamente.

Si accedemos a la sección de ‘Checks’ dentro del asset en cuestión podemos ver la información relativa a dicho check, así como a sus metadatos, tal y como se muestra en la siguiente Figura:



The screenshot shows the Dagster Checks interface. On the left, there's a sidebar with 'Checks (1)' and a search bar. A check named 'check_estandarizacion_islas' is listed with a green checkmark and 'Succeeded'. The main panel shows the details for 'check_estandarizacion_islas'. It has tabs for 'Overview', 'Execution history', and 'Automation history'. The 'Overview' tab is active, showing 'About' (a non-blocking asset check with no description), 'Latest execution' (Passed at 10 mar, 12:07), and 'Metadata' (categorias_detectadas: 3, categorias_esperadas: 3, mensaje: Si hay nombres inconsistentes, ggplot creará leyendas duplicadas., principio_gestalt: Similitud (Evitar fragmentación visual)).

Figura 2. Check `check_estandarizacion_islas` validado correctamente en Dagster.

Con esto se confirma que el asset y el check funcionan correctamente dentro del proyecto y que Dagster realiza la validación de forma automática al materializar el asset.

Forzado del fallo del check y análisis del resultado

Para comprobar el funcionamiento real del sistema de Asset Checks, se modifica el asset `islas_raw` añadiendo de forma intencionada una fila con el nombre de la isla escrito en minúsculas (“tenerife”). De esta forma se introduce una inconsistencia en los datos con el objetivo de provocar el fallo del check.

Tras guardar los cambios y volver a materializar el asset en Dagster, se observa que el check deja de validarse correctamente y pasa a estado fallido.



The screenshot shows the Dagster asset 'islas_raw'. It displays the latest event as '12 seconds ago'. The 'Asset checks' section shows a red 'X' and '0 / 1 Passed'. The 'Partitions' and 'Automation' sections show a dash. The 'Materialized' section shows '10 mar, 12:46'.

Figura 3. Materialización del asset `islas_raw` con el check fallido (0/1 Passed).

Como se puede ver en la interfaz, el sistema indica que el check ha fallado, mostrando el estado “0/1 Passed”. Esto confirma que la condición definida en el check ya no se cumple.

Si accedemos al detalle del check dentro de la pestaña “Checks”, podemos ver información más concreta sobre el motivo del fallo.

The screenshot shows a web interface with tabs: Overview, Events, Checks, and Lineage. The 'Checks' tab is active, showing a list of checks on the left and details for 'check_estandarizacion_islas' on the right.

Checks (1)

Filter checks

check_estandarizacion_islas
Failed

Execute all

check_estandarizacion_islas

Overview Execution history Automation history

About

This is a **non-blocking** asset check. ⓘ

No description provided

Latest execution

Evaluation result	Timestamp	Target materialization	Partition
Failed	10 mar, 12:46	10 mar, 12:46	—

Metadata

categorias_detectadas	4
categorias_esperadas	3
mensaje	Si hay nombres inconsistentes, ggplot creará leyendas duplicadas.
principio_gestalt	Similitud (Evitar fragmentación visual)

Figura 4. Detalle del check `check_estandarizacion_islas` tras el fallo, mostrando las categorías detectadas y las esperadas.

En este caso, el sistema detecta 4 categorías originales, mientras que tras la normalización deberían existir únicamente 3. Esto se debe a que ahora aparecen tanto “Tenerife” como “tenerife” como categorías distintas, lo que rompe la consistencia en los nombres.

Con esta prueba se demuestra que el check funciona correctamente, ya que es capaz de detectar automáticamente inconsistencias en los datos que podrían generar problemas en la visualización, como leyendas duplicadas o fragmentación visual.

Implementación de checks en el proyecto de rentas

En este apartado se realiza un análisis del proyecto de visualización de rentas desarrollado anteriormente, con el objetivo de incorporar controles de calidad similares a los vistos en el ejemplo anterior. La idea es aplicar Asset Checks no solo en la fase de carga de datos, sino también en las etapas de transformación y visualización.

Para ello, se revisan posibles problemas que puedan afectar a los datos o a las visualizaciones y se plantean comprobaciones para detectarlos automáticamente. Así, el pipeline no solo genera los resultados, sino que también verifica que todo sea coherente antes de darlo por válido.

Lo primero es modificar el fichero definitions.py para ajustarlo al proyecto lab_renta.py.

Una vez incorporados los checks al proyecto, se ejecuta el pipeline completo en Dagster para comprobar su funcionamiento. En la siguiente Figura se muestra el pipeline general del proyecto, donde se puede observar que todos los checks definidos se han ejecutado correctamente.

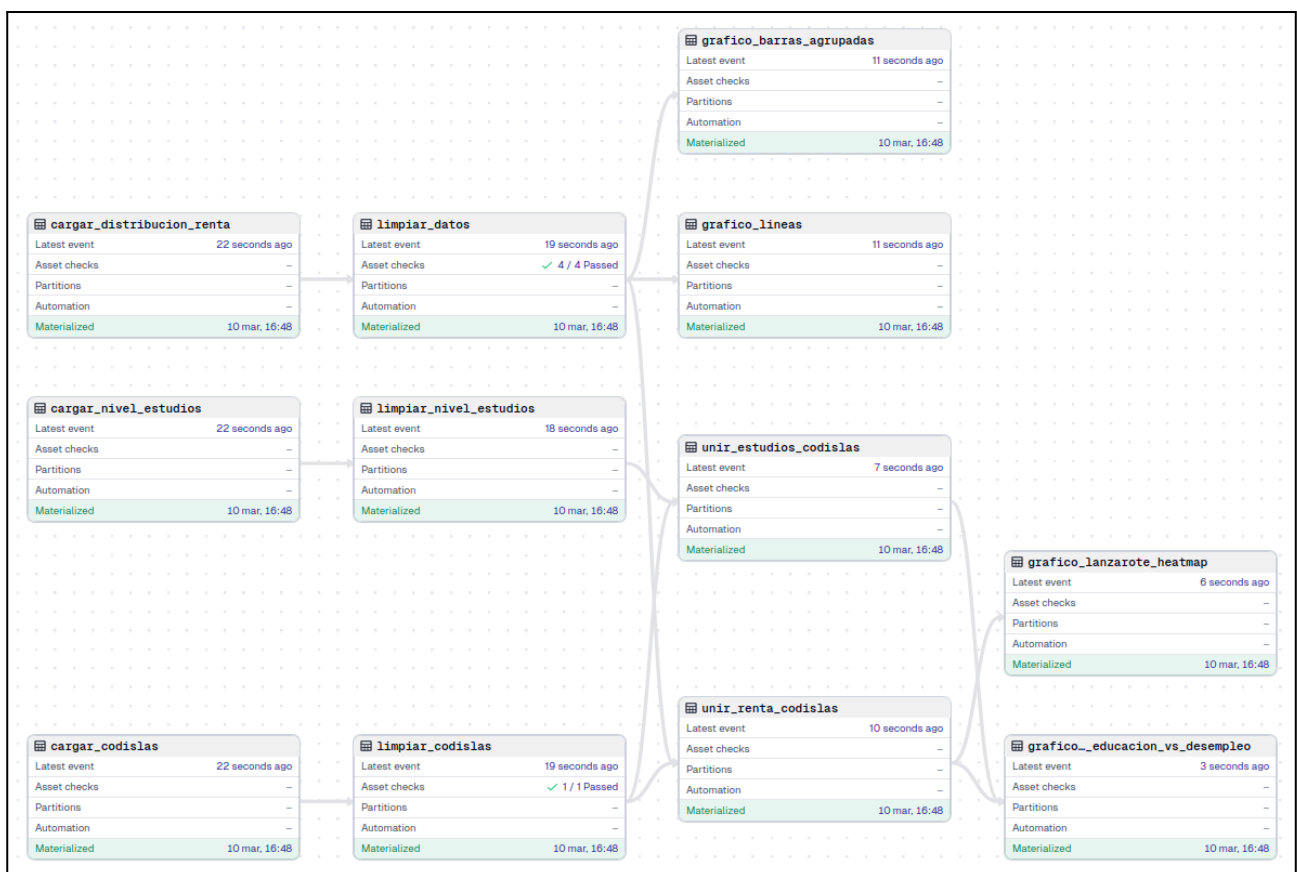


Figura 5. Vista general del pipeline en Dagster con todos los checks superados.

Verificación del formato de los códigos territoriales

Este check valida que los códigos de municipio tengan exactamente cinco dígitos numéricos.

Si los códigos no tienen el formato correcto, las uniones entre tablas podrían fallar o generar municipios sin correspondencia. En este proyecto, los códigos territoriales son la clave que permite relacionar la información de renta con la información geográfica, por lo que su coherencia es fundamental.

El resultado muestra que no se detectó ningún código inválido, lo que confirma que los datos están correctamente estandarizados y preparados para realizar las uniones de los datasets sin pérdidas de información.

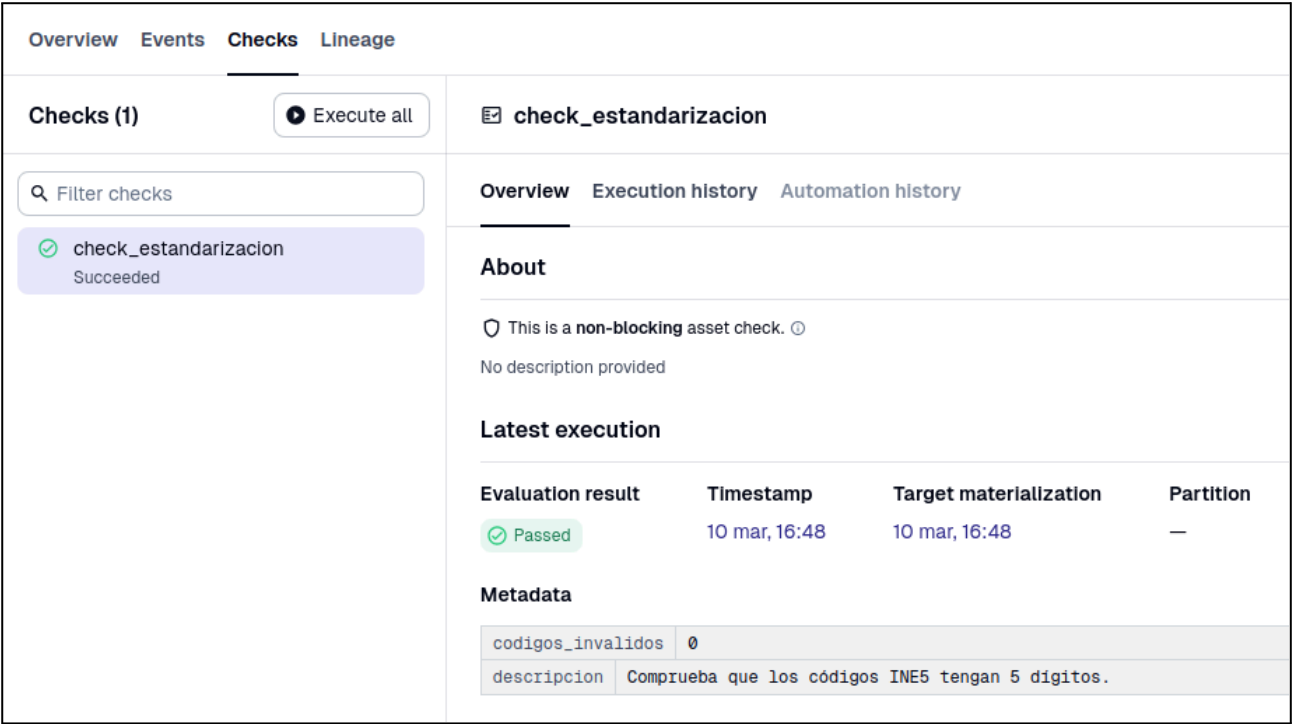


Figura 6. Check de validación del formato de los códigos territoriales.

Verificación de ausencia de valores nulos en datos clave

En este caso se comprueba que no existan valores nulos en los campos esenciales para el análisis: el año, el tipo de renta y el valor porcentual correspondiente.

La lógica de este check es que una visualización no puede construirse correctamente si falta alguno de estos elementos. Por ejemplo, un valor porcentual vacío impediría representar un punto en la gráfica, y la ausencia del año rompería la serie temporal.

El resultado indica que el número total de valores nulos en estos campos es cero, por lo que los datos están completos y listos para su análisis.

Overview

Events

Checks

Lineage

Checks (4)

Execute all

Filter checks

check_cardinalidad

Succeeded

check_continuidad

Succeeded

check_eje_cero

Succeeded

check_nulos_criticos

Succeeded

check_nulos_criticos

Overview

Execution history

Automation history

About

This is a **non-blocking** asset check.

No description provided

Latest execution

Evaluation result	Timestamp	Target materialization	Partition
Passed	10 mar, 16:48	10 mar, 16:48	—

Metadata

descripcion	Comprueba que no haya nulos en columnas clave.
total_nulos	0

Figura 7. Check de ausencia de valores nulos superados correctamente.

Verificación de continuidad temporal

Este check comprueba que la serie temporal contenga varios años y no esté limitada a un único periodo.

En concreto, se ha establecido como criterio que existan al menos cinco años distintos, ya que con menos periodos el análisis temporal perdería sentido y podría llevar a interpretaciones poco sólidas. En el conjunto de datos se detectan nueve años diferentes (de 2015 a 2023), por lo que se supera ampliamente el umbral mínimo definido.

El resultado confirma que la serie temporal es suficientemente amplia para analizar tendencias de forma coherente.

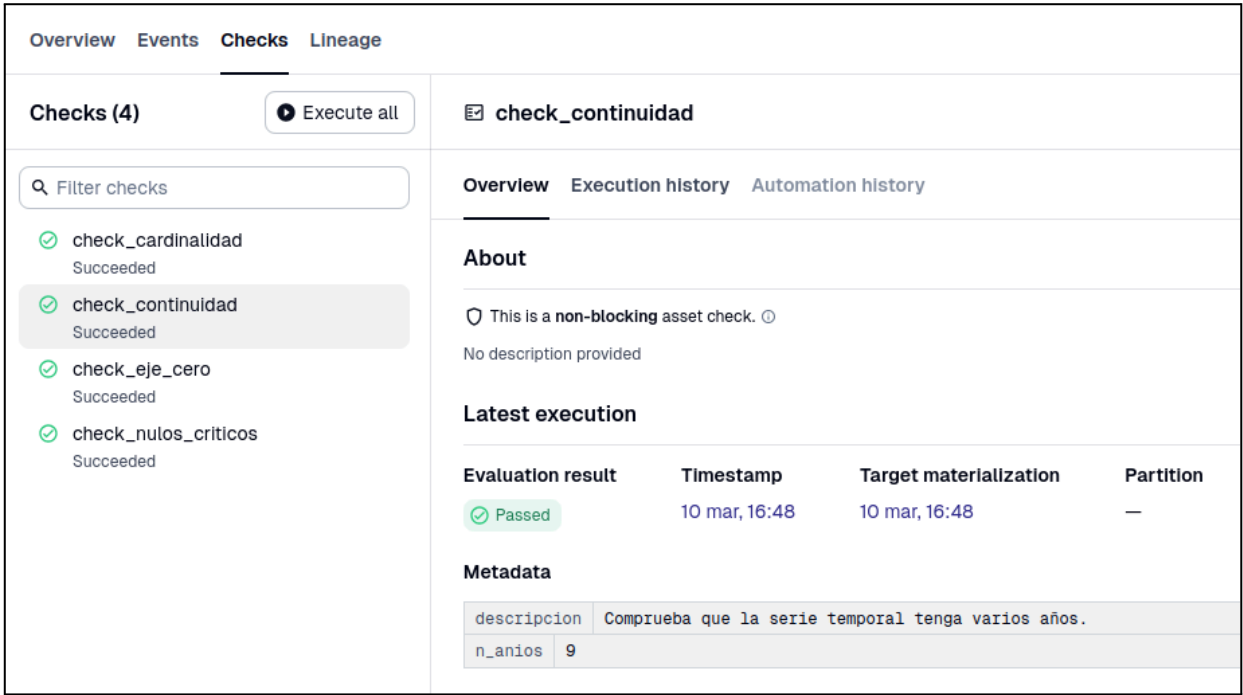


Figura 8. Check de continuidad temporal validado correctamente.

Control del número de categorías representadas

En este caso se controla que el número de tipos de renta representados en los gráficos no sea excesivo.

Se ha establecido como criterio que el número de categorías no supere seis grupos distintos. Este límite se fija porque, a partir de cierto número de categorías, la gráfica comienza a sobrecargarse y resulta más difícil comparar los valores de forma clara. Cuando hay demasiados grupos, la interpretación pierde precisión.

En los datos utilizados se detectan cinco categorías diferentes, por lo que se cumple la condición definida y la representación se considera perfectamente comprensible.

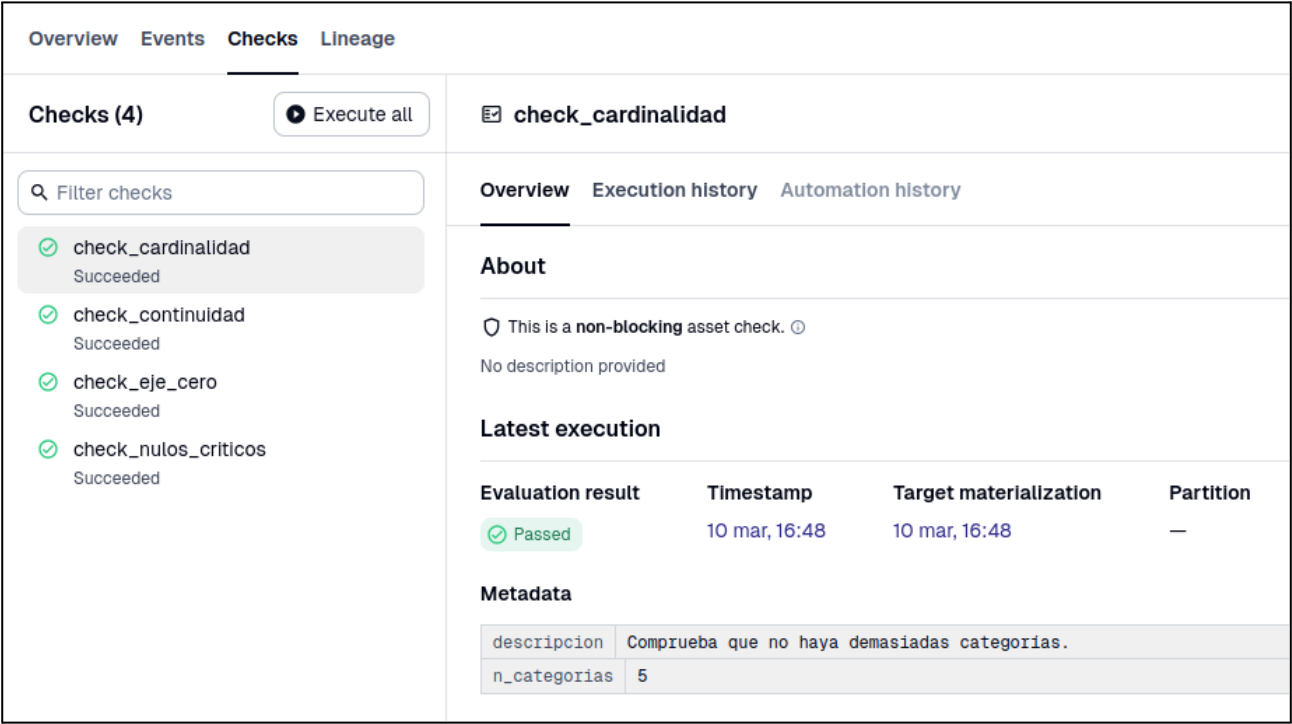


Figura 9. Check de control del número de categorías superado correctamente.

Verificación de coherencia del eje vertical

Por último, se comprueba que todos los valores porcentuales sean positivos, lo que permite que el eje vertical pueda comenzar en cero sin alterar la interpretación del gráfico. Si hubiera valores negativos, el eje cambiaría y la gráfica podría dar una impresión equivocada.

En este caso, el valor mínimo detectado es 2,3%, por lo que todos los datos son positivos y la gráfica puede representarse desde cero sin generar distorsiones.

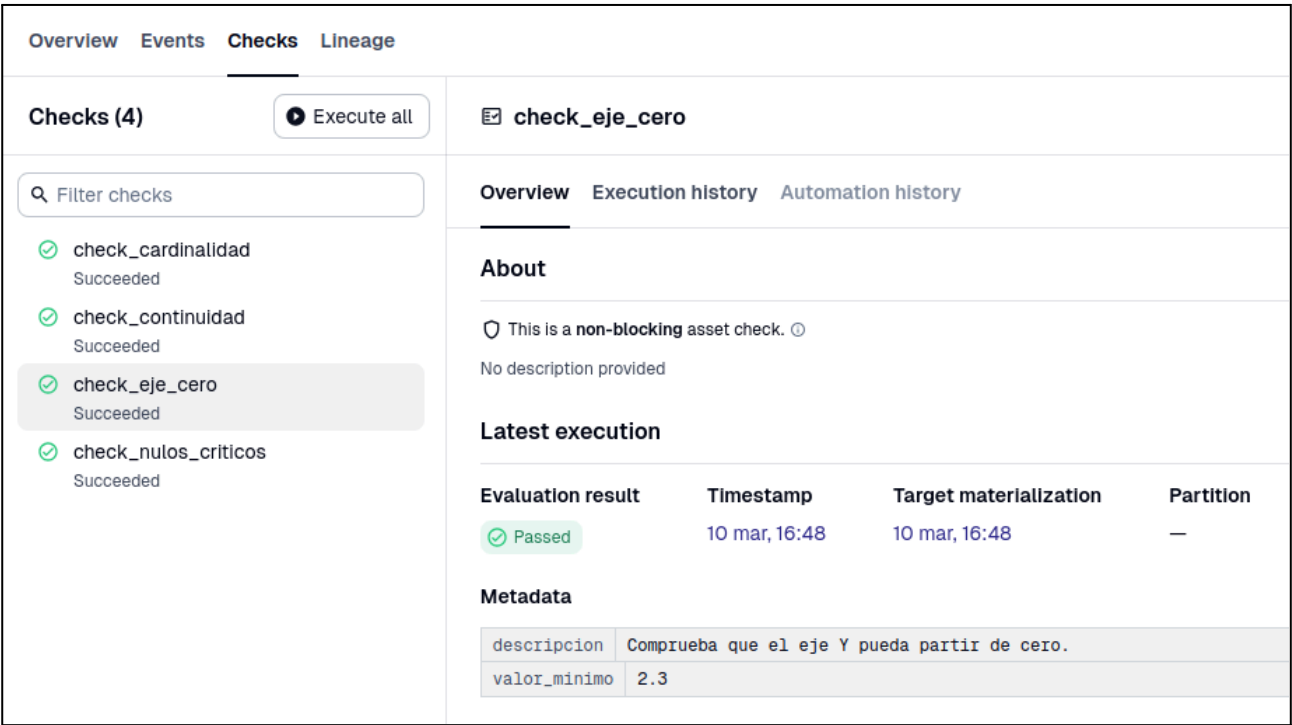


Figura 10. Check de validación del eje vertical superado correctamente.

En conjunto, estos checks añaden un control extra al proyecto. No solo se comprueba que los datos estén bien preparados, sino también que las gráficas que se generan tengan sentido y sean fáciles de interpretar.

Así, el pipeline no se limita a crear visualizaciones, sino que también revisa automáticamente que el resultado sea coherente antes de considerarlo correcto.

Forzado de fallo de un check

Para verificar que los checks implementados detectan correctamente errores en los datos, se ha provocado de forma intencionada un fallo que incumpla una de las verificaciones.

En este caso, se ha reducido temporalmente el umbral permitido de categorías, pasando de seis a dos. Sin embargo, el conjunto de datos contiene cinco tipos de renta distintos, por lo que la condición definida deja de cumplirse.

Tras ejecutar nuevamente el pipeline en Dagster, el check aparece marcado como no superado, indicando que el número de categorías excede el límite establecido.

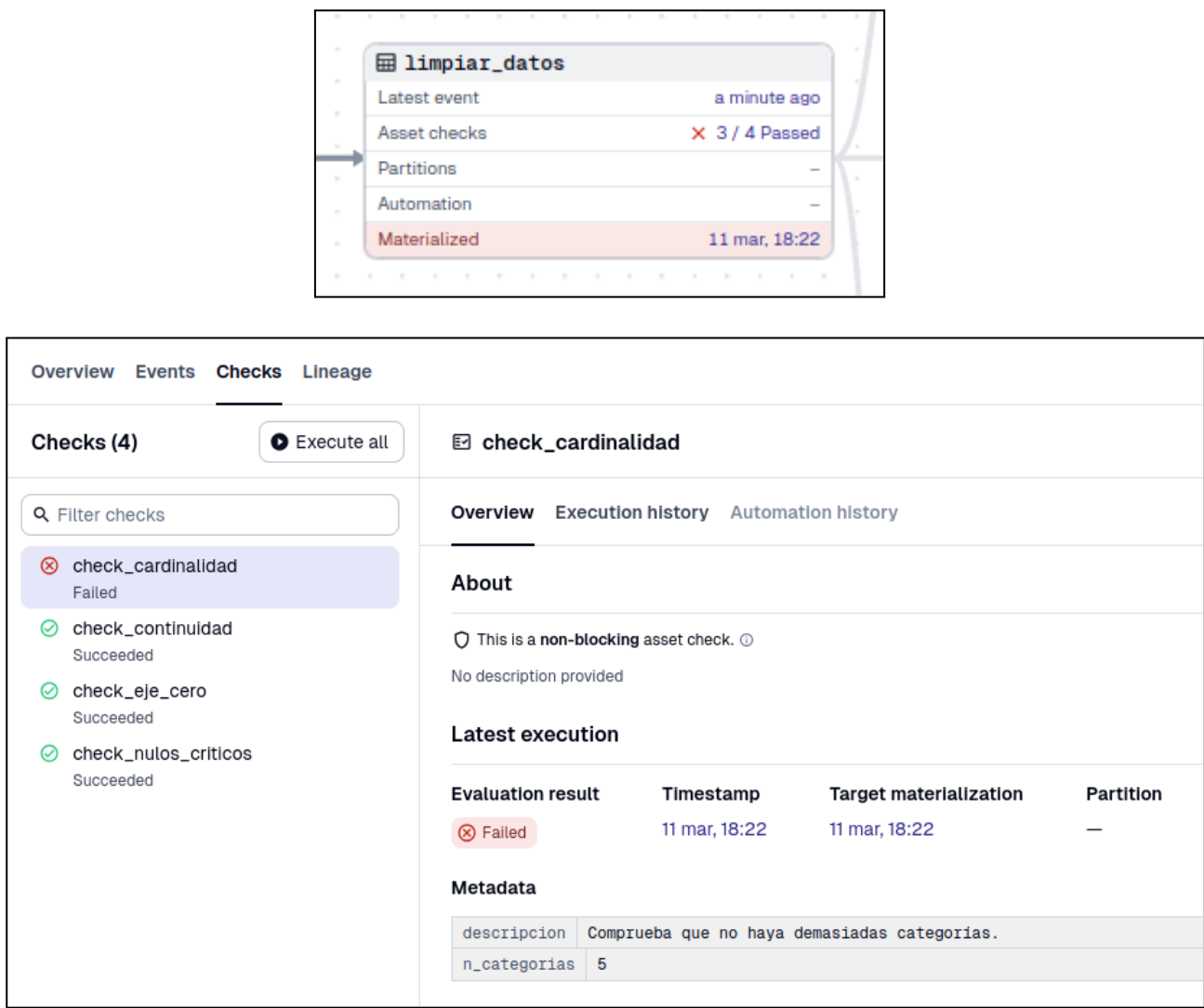


Figura 11. Check de comprobación del número de categorías no superado tras reducir el umbral permitido.

Este resultado demuestra que los checks funcionan correctamente y que el pipeline detecta automáticamente cuando no se cumple la condición establecida.